# Results

## Fixture Release Packet I

The fixture release packet contains fourteen segments spanning four classification levels: UNCLASSIFIED (ten segments), CUI (one segment), SECRET (two segments), and TOP_SECRET (one segment). Four segments carry source controls (drawn from the HUMINT, SIGINT, and IMINT disciplines). Twenty-two redaction decisions are applied across those four segments, using all five bounded reasons: `source_identity`, `operational_detail`, `time_place_selector`, `legal_privilege`, and `privacy`.

# Fixture Release Packet I

The audit produces:

## Fixture Release Packet I

- **Releasable**: true (no error-level findings after redaction).
- **Release safety score**: weighted by error count, warning count, and mosaic risk.
- **Redaction coverage**: 1.0 (all sensitive segments have at least one decision).
- **Mosaic risk score**: residual markers normalized by segment and pattern count.
- **Findings**: warning-level findings for residual markers in sanitized text.

## Fixture Release Packet I

The canonical run's measured values are bound to `output/reports/redaction_audit.json` by the manuscript-binding tests:

# Fixture Release Packet I

| Metric | Measured value |
|---|---|
| Segment count | 14 |
| Redaction decision count | 22 |
| Reviewer record count | 3 |
| Release safety score | 0.798 |
| Redaction coverage | 1.0 |
| Mosaic risk score | 0.012 |
| Findings | 2 warning-level |
| Releasable | true |

# Fixture Release Packet I

| Metric | Measured value |
|---|---|
| Final release recommended | false |
| Approvals | 3 |

## Fixture Release Packet I

Because the intelligence policy blocks on warnings and the fixture retains two warning-level residual markers, `final_release_recommended` is false even though the packet is releasable and the review gate is approved.

## Source-Safe Redaction Ledger I

The redaction ledger records each decision with:

# Source-Safe Redaction Ledger I

- A decision ID derived from the SHA-256 of the segment ID, span, reason, and replacement.
- The segment ID, start, end, span length, reason, and replacement.
- A `valid_span` flag indicating whether the span falls within the segment text.
- A `source_span_sha256` hash of the redacted text—present only for valid spans, absent for invalid or orphan decisions.

## Source-Safe Redaction Ledger I

The ledger never exposes source text. It provides reproducible audit evidence without disclosure risk.

The normal analysis stage writes this evidence to `output/data/release_ledger.json`. Its companion `output/reports/redaction_audit.json` contains policy outcomes, review-gate status, findings, scores, counts, and paragraph metadata. Both canonical files omit narrative text, review rationales, and reviewer identities; the comprehensive sanitized packet is evaluated in memory rather than serialized to the public output tree.

# Segment Hash Manifest I

The hash manifest records, for each segment:

# Segment Hash Manifest I

- `source_sha256`: SHA-256 of the original segment text.
- `public_sha256`: SHA-256 of the redacted segment text.

## Segment Hash Manifest I

This allows downstream verification that the public release matches the audited version without comparing source text directly.

## Review Gate I

The release gate requires three reviewer roles: originator, classification reviewer, and release authority. Each reviewer provides a non-empty rationale. The fixture reviews all approve, yielding: The complementary input is also enforced: missing roles or empty rationales mean the release gate fails closed.

## Review Gate I

- **Approved**: true.
- **Approval count**: 3.
- **Required roles present**: classification_reviewer, originator, release_authority.
- **Findings**: none.

## Review Gate I

The `final_release_recommended` flag is true only when the packet is releasable, the review gate is approved, and no blocking warnings remain.

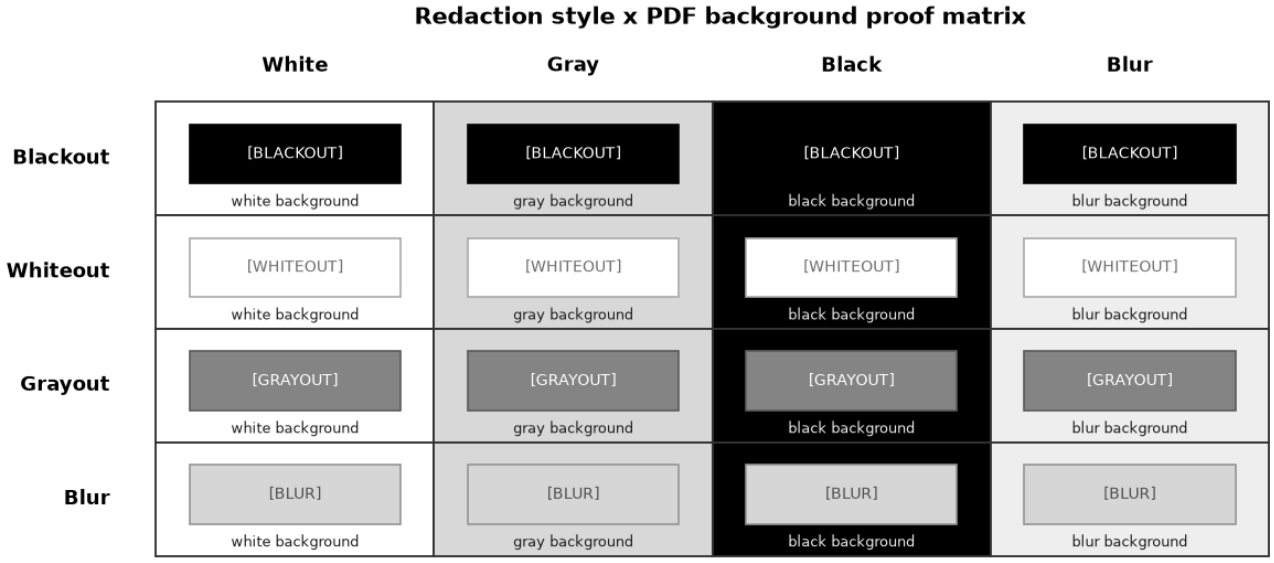
## Visual Proof Matrix I

The development proof matrix produces sixteen base PDFs (four redaction styles $\times$ four backgrounds) and, when steganography is enabled, sixteen companion steganography PDFs with hash manifests. Each variant records:

# Visual Proof Matrix I

- Base PDF filename, byte size, and SHA-256.
- Steganography PDF filename, byte size, and SHA-256.
- Hash manifest filename and SHA-256.
- Kmyth sidecar count and filenames (when Kmyth is available).

# Visual Proof Matrix I




All sixteen variants apply the identical source-safe redaction decisions;
only the visual token and page background differ.

Styles: Blackout: [BLACKOUT] | Whiteout: [WHITEOUT] | Grayout: [GRAYOUT] | Blur: [BLUR]

# Visual Proof Matrix I

Figure 2: Development proof matrix: four redaction styles (blackout, whiteout, grayout, blur) rendered across four PDF backgrounds (white, gray, black, blur); every one of the sixteen cells applies the identical source-safe redaction decisions.

## Visual Proof Matrix I

fig. 2 renders the sixteen style-background combinations; only the visual token and the page background differ between cells.

# Kmyth TPM Sidecar Production I

When Kmyth tools are runnable and a TPM backend is available, each variant produces two `.ski` sidecars:

# Kmyth TPM Sidecar Production I

1. `{variant_id}.hashes.json.ski` — the hash manifest sealed against the TPM storage hierarchy.
2. `{variant_id}_steganography.pdf.ski` — the steganography PDF sealed against the TPM storage hierarchy.

## Kmyth TPM Sidecar Production I

The `.ski` files are ASCII-armored and contain PCR selections, policy or-values, the storage key public area, and the sealed data object. Unsealing requires the same TPM and platform configuration.

In the verified run, all sixteen variants produced both sidecars, yielding thirty-two `.ski` files total. The kmyth-seal binary's FlushContext patch ensured that consecutive seal invocations did not exhaust the swtpm transient object slots.

## Residual Risk Detection I

The residual-risk detector scans sanitized text for common public-release leaks:

# Residual Risk Detection I

| Pattern | Regex |
|---|---|
| Email address | `\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b` |
| IPv4 address | `\b(?:\d{1,3}\.){3}\d{1,3}\b` |
| Coordinate pair | `\b-?\d{1,2}\.\d{3,},\s*-?\d{1,3}\.\d{3,}\b` |
| Controlled dissemination | `\b(?:NOFORN\|ORCON\|REL\s+TO)\b` |
| Collection discipline | `\b(?:HUMINT\|SIGINT\|IMINT\|MASINT\|OSINT)\b` |
| Compartment marker | `\b(?:SCI\|TS_SCI\|TOP\s+SECRET)\b` |
| ISO calendar date | `\b20\d{2}-\d{2}-\d{2}\b` |
| Sensitive markers | `HUMINT`, `SIGINT`, `source`, `selector`, `location` |

## Residual Risk Detection I

Each detected marker generates a warning finding. The mosaic risk score aggregates residual markers across all segments.

# Discussion

## Discussion I

The exemplar demonstrates that disclosure control can be decomposed into orthogonal concerns: text-level audit, visual presentation, steganographic provenance, and hardware-backed sealing. Each concern is independently configurable and verifiable.

The visual proof matrix confirms that blackout, whiteout, grayout, and blur treatments produce equivalent source-safe outputs—only the visual token differs. The steganography layer adds provenance without altering the redaction decisions. Kmyth TPM sealing adds a hardware binding that ensures sealed sidecars can only be unsealed on the same platform configuration.

## Discussion I

The mssim-to-swtpm protocol proxy is a necessary bridge on macOS, where no hardware TPM exists. The proxy handles three protocol mismatches: (1) platform commands use different command numbers, (2) the data channel wraps TPM commands in a 9-byte mssim header, and (3) swtpm's transient object slots are limited and must be flushed between seal invocations.

The FlushContext patch to kmyth-seal is a critical fix for batch sealing workflows. Without it, the second kmyth-seal invocation fails with "out of memory for object contexts" because the storage key from the first invocation remains loaded in the TPM's transient object table.

# Limitations I

- The fixture data is invented; real release packets may require additional classification taxonomies and review-role policies.
- The swtpm software TPM does not provide hardware-level tamper resistance; production deployments should use a hardware TPM.
- The mssim proxy adds latency to each TPM command; batch sealing of thirty-two sidecars takes approximately thirty seconds.
- PDF password encryption is optional and uses AES-256; the password is not stored in the variant matrix.